Doc bibliothèque Pic +

Idée principale de Pic+ :

- avoir accès à des fonctions basiques et facilitées
- rendre le code plus lisible
- rendre le développement plus rapide dans le cadre de la micro-informatique à l'IG2I

Informations de préambule :

- Si un mot est en *italique*, il a donc été défini comme mot-clé du langage.
- Si un mot est en **gras**, c'est que c'est du code ou une variable.

Sommaire :

1. Installation dans le projet

2. Modules

2.1. Commun

type **octet**

type **t_port**

fonction **perte_temps**

fonction **confINT**

fonction **DisableADConvert**

2.2. Virtual Port

type **t_portID**

type **t_VirtualPort**

fonction **beginVPort**

fonction **addBitsVPort**

fonction **readVPort**

fonction **readVPin**

fonction **writeVPort**

fonction **writeVPin**

fonction **trisVPort**

fonction **trisVPin**

2.3. Analog

fonction **confAnalogTomV**

fonction **analogTomV**

fonction **confAnalog**

fonction **analogRead**

2.4. TMR0

fonction **confTMR0**

fonction **confTMR0ms**

fonction **confTMR0us**

fonction **resetTMR0**

2.5. TMR1

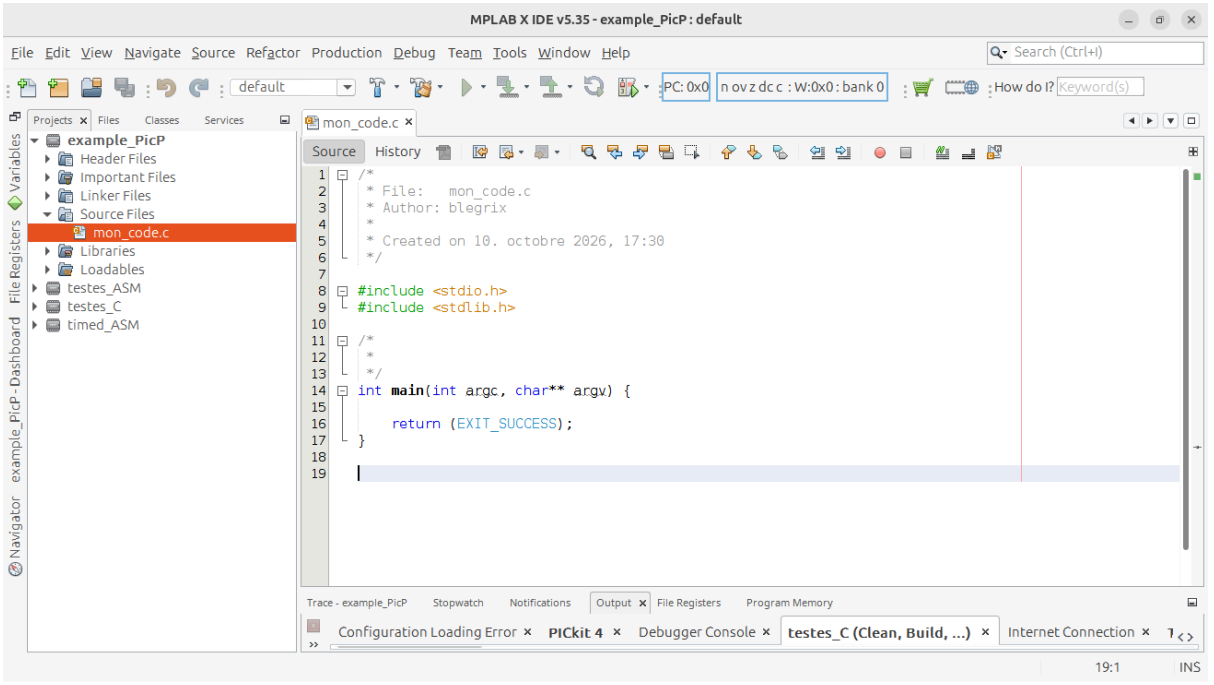
fonction **confTMR1**

fonction **confTMR1us**

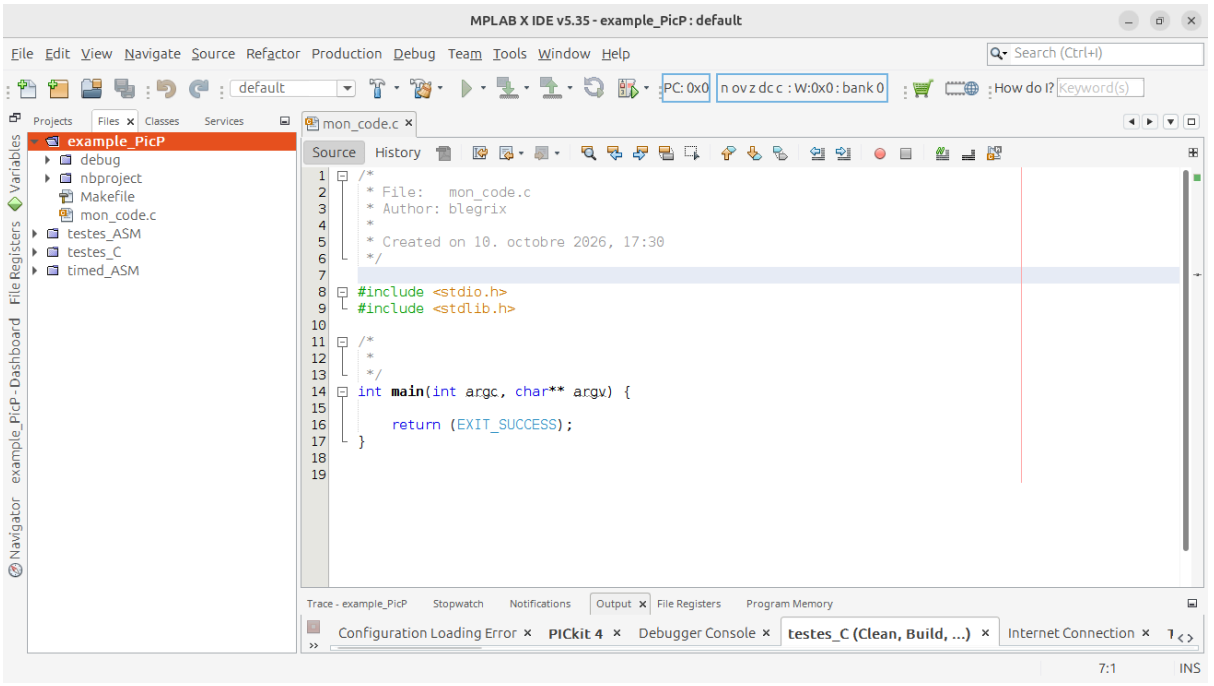
fonction **resetTMR1**
2.6. TMR2
fonction **confTMR2**
fonction **confTMR2us**
fonction **resetTMR2**
2.7. TMR3
fonction **confTMR3**
fonction **confTMR3us**
fonction **resetTMR3**
2.8. CCPX
2.9. aff7seg
fonction **conf7seg**
fonction **convert7seg**
fonction **write7seg**
fonction **conf7segVP**
2.10. KeyPad
fonction **confKeyPadVP**
fonction **confKeyPad**
fonction **keyPadRead**
fonction **isKeySet**
fonction **keyBitmapChange**
fonction **keyBitmapToInt**
fonction **keyBitmapToIntPL**
fonction **keyBitmapToIntPG**
2.11. Sequential
fonction **initSequence**
fonction **addSequenceStep**
fonction **SequenceStep**
fonction **SequanceSetPos**
2.12. LCD
fonction **LireTrameLCD**
fonction **EcrireTrameLCD**
fonction **confLCD**
fonction **printIntLCD**
fonction **clearLCD**
fonction **gotoLCD**
fonction **printLCD**
fonction **confCurseurLCD**
fonction **moveLCD**

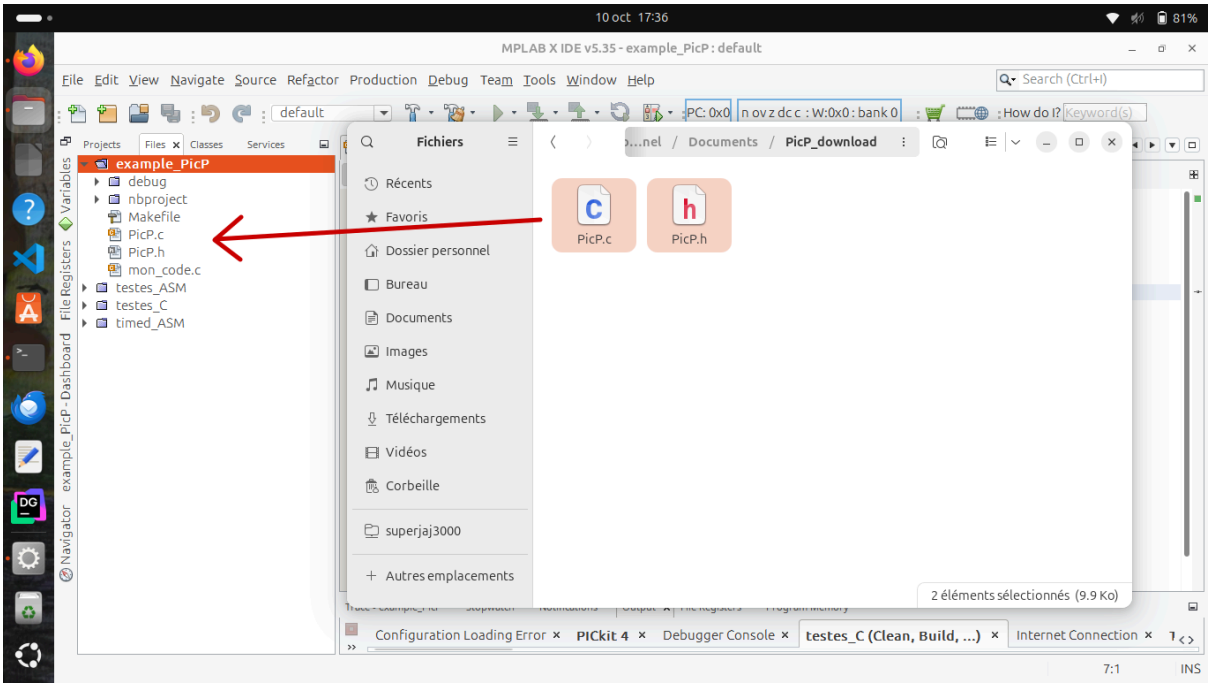
1)Installation dans le projet

Nous allons montrer dans un projet imaginaire « example_PicP » :

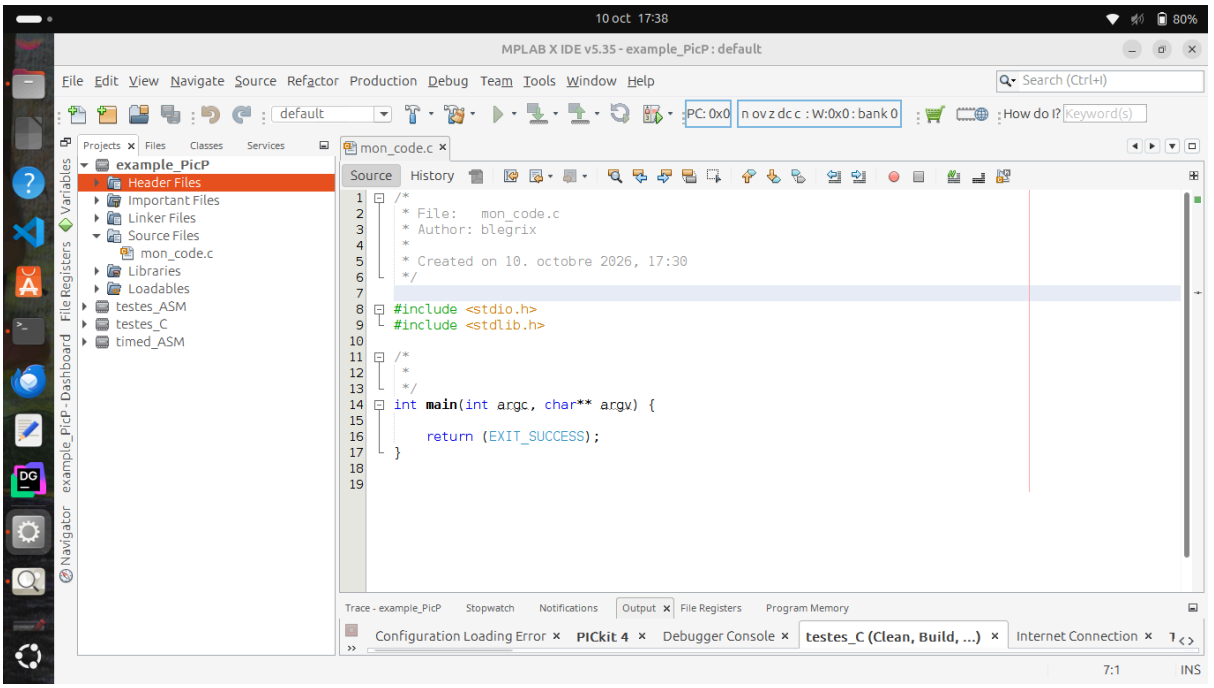


1. Aller dans Files et ajouter les fichiers PicP.h et PicP.c à la racine.

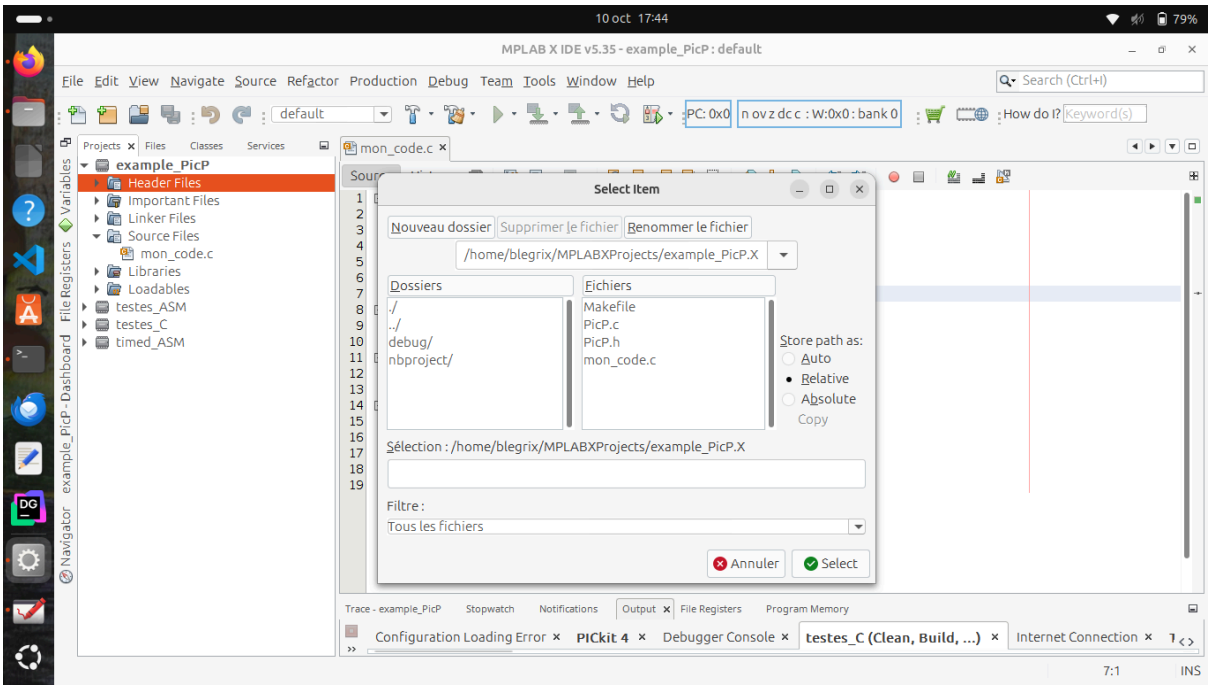




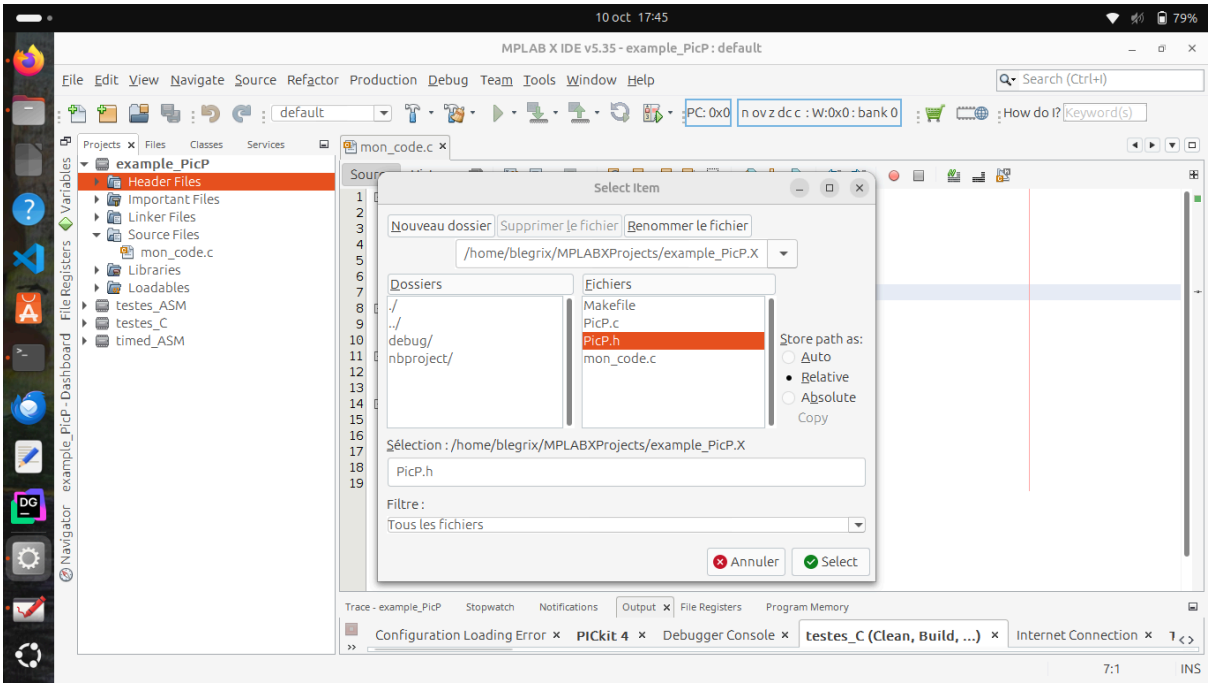
2. Aller dans Projects.



3. Ajouter au projet PicP.h dans les Header Files :
   3.1. Clic droit sur « Header Files ».
   3.2. Clic sur « Add Existing Item... ».

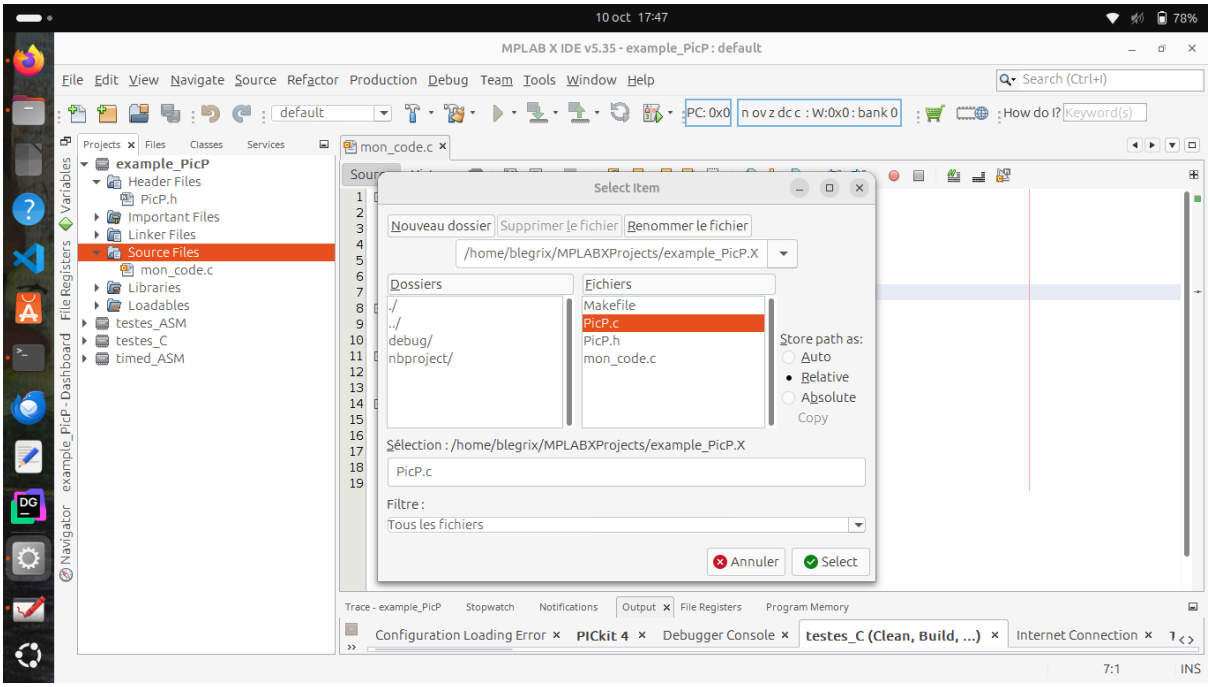


3.3. Clic sur PicP.h.

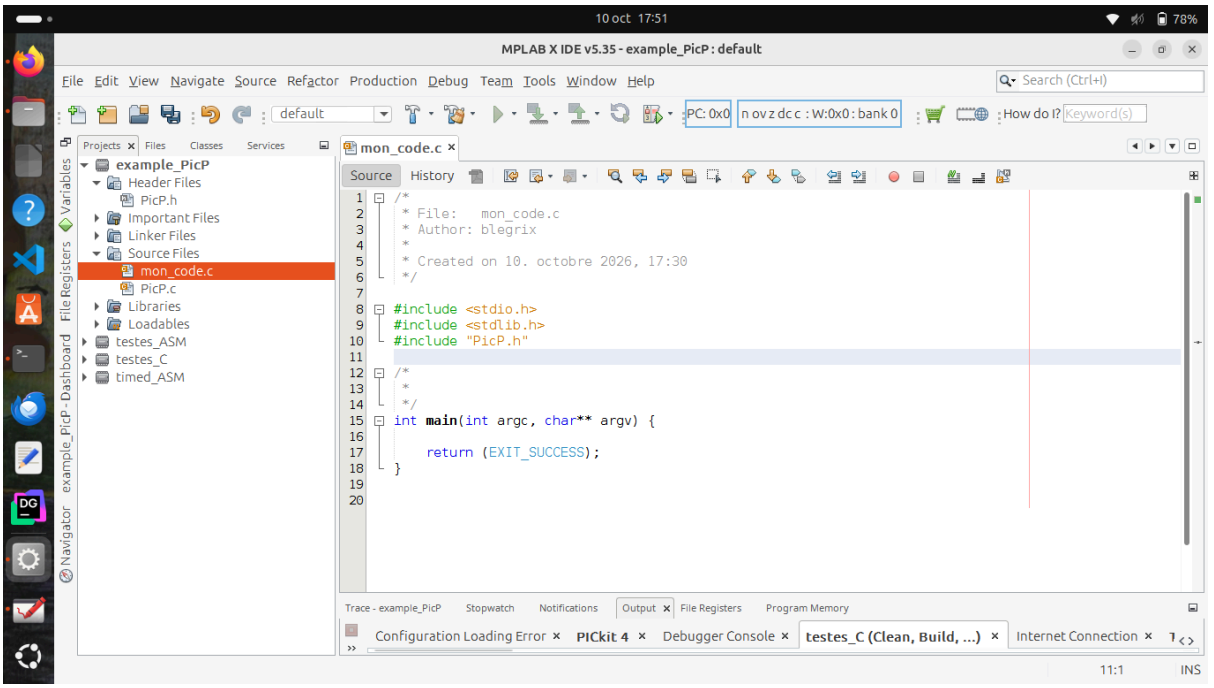


3.4. Puis clic sur Select.

4. On fait la même chose avec PicP.c dans Source Files.



5. On n'a plus qu'à importer la lib dans notre code source :
   5.1. Aller dans votre fichier votre_code.c.
   5.2. Avec les inclusions, ajouter la ligne :
   #include "PicP.h"



6. La lib est installée, vous pouvez donc utiliser les modules ci-dessous en les activant par décommentaire de leur définition dans PicP.h.

2)Modules

Chaque module hormis commun doit être activé dans PicP.h en décommentant le :

#define [nom_du_module]_module

Commun

Le module commun contient des fonctions utiles en permanence comme :

Type **octet** : c'est un unsigned char qui ne représente qu'un octet non signé.

Type **t_port** : c'est un volatile unsigned char qui représente les PORTX et TRISX pour les affecter.

void perte_temps(int tempo);
Effectue une perte de temps processeur proportionnelle à (tempo).

Exemple : vous avez besoin d'un délai entre deux actions :

```
C/C++
prog{
    action1();
    tempo(200);
    action2();
}
```

void confINT(octet EN_INT0, octet EN_INT1, octet EN_INT2);
Active ou non les interruptions **INT0**, **INT1** et **INT2**.

Variables ->
EN_INTx -> *enable* = l'interruption **INTx** est configurée et allumée
disable = l'interruption **INTx** est éteinte.

Exemple : vous avez besoin de l'interruption physique 0 mais pas des interruptions 1 & 2 :

```
C/C++
init{
    confINT(enable, disable, disable);
}
```

void DisableADConvert();
Désactive les entrées analogiques pour pouvoir utiliser les ports analogiques en numérique.

Virtual Port

Ce module permet l'usage de « ports virtuels », ces ports sont composés de broches de multiples ports.
Si ce module est activé, il débloque les fonctions ayant le marqueur *(VP)*.
De plus, l'acronyme VP = Virtual Port sera utilisé.
Le standard VP indique que les modules utilisant des VP doivent configurer automatiquement le TRIS des VP utilisés.

Type t_portID : c'est un type compact représentant un identifiant de port selon le tableau ci-dessous.

Port	Définition dans Pic+	Valeur
PORTA	PA	0
PORTB	PB	1
PORTC	PC	2
PORTD	PD	3
PORTE	PE	4

Type t_VirtualPort : c'est le type qui contient un VP, il y a des VP prédéfinis :

Définition dans Pic+	Port virtualisé
VPA	PORTA
VPB	PORTB
VPC	PORTC
VPD	PORTD
VPE	PORTE

void beginVPort(t_VirtualPort *VPort);
Nettoie le VP *VPort pour pouvoir commencer à lui ajouter des bits.

void addBitsVPort(t_VirtualPort *VPort, t_portID Port, octet startID, signed char length);
Ajoute la rangée de bits sur le port **Port** en commençant par **startID** sur **length** bits à **VPort**.

octet readVPort(t_VirtualPort *VPort);
Lit et retourne la valeur numérique du port virtuel ***VPort**.

octet readVPin(t_VirtualPort *VPort, octet pin);
Lit et retourne la valeur binaire (0 ou 1) de la broche **pin** du port virtuel ***VPort**.

void writeVPort(t_VirtualPort *VPort, octet data);
Écrit la valeur **data** sur l'ensemble des broches du port virtuel ***VPort**.

void writeVPin(t_VirtualPort *VPort, octet data, octet pin);
Écrit l'état **data** (0 ou 1) sur la broche **pin** du port virtuel ***VPort**.

void trisVPort(t_VirtualPort *VPort, octet tris);
Configure la direction (entrée/sortie) via **tris** pour l'ensemble des broches du port virtuel ***VPort**.

void trisVPin(t_VirtualPort *VPort, octet tris, octet pin);
Configure la direction **tris** de la broche **pin** du port virtuel ***VPort**.

Analog

Ce module permet de gérer les entrées analogiques (CAN/ADC) et la conversion en millivolts (mV).
Macros de sélection des canaux analogiques (Pins) :
• **AN0** à **AN7** (valeurs binaire de 0b000 à 0b111) désignent les canaux d'entrée analogique 0 à 7.

#define defaultConfAnalogTomV 21.8,5085.5
Configuration par défaut de la conversion tension/binaire (tension min = 21.8 mV, tension max = 5085.5 mV).

void confAnalogTomV(double min, double max);
Redéfinit la configuration de calcul de **analogTomV** en fonction des tensions minimale (min) et maximale (max) exprimées en mV.

int analogTomV(int analog);
Convertit la valeur binaire lue via **analogRead** en une valeur de tension équivalente en millivolts (mV) selon la configuration définie.

void confAnalog(octet mode_10bit, octet PCFG);
Configure les entrées analogiques du microcontrôleur. **mode_10bit** définit la résolution de lecture (10-bit ou 8-bit) et **PCFG** configure le registre de sélection des broches analogiques/numériques.
Rappel de PCFG :
rappel de PCFG:

Définition rapide	PCFG<3:0>	AN7	AN6	AN5	AN4	AN3	AN2	AN1	AN0	VREF+	VREF-
PCFG_ALL	0b0000	A	A	A	A	A	A	A	A	VDD	VSS
	0b0001	A	A	A	A	VREF+	A	A	A	AN3	VSS
PCFG_A0_A4	0b0010	D	D	D	A	A	A	A	A	VDD	VSS
	0b0011	D	D	D	A	VREF+	A	A	A	AN3	VSS
PCFG_A0_1_3	0b0100	D	D	D	D	A	D	A	A	VDD	VSS
	0b0101	D	D	D	D	VREF+	D	A	A	AN3	VSS
PCFG_no_port	0b011x	D	D	D	D	D	D	D	D	—	—
	0b1000	A	A	A	A	VREF+	VREF-	A	A	AN3	AN2
PCFG_A0toA5	0b1001	D	D	A	A	A	A	A	A	VDD	VSS
	0b1010	D	D	A	A	VREF+	A	A	A	AN3	VSS
	0b1011	D	D	A	A	VREF+	VREF-	A	A	AN3	AN2
	0b1100	D	D	D	A	VREF+	VREF-	A	A	AN3	AN2
	0b1101	D	D	D	D	VREF+	VREF-	A	A	AN3	AN2
PCFG_A0	0b1110	D	D	D	D	D	D	D	A	VDD	VSS
	0b1111	D	D	D	D	VREF+	VREF-	D	A	AN3	AN2

int analogRead(octet Pin);
Effectue une conversion analogique-numérique et renvoie la valeur binaire lue sur le canal passé en paramètre (ex: **AN0**).

Exemple : lire la tension en mV sur le canal AN0 et effectuer une action si elle dépasse 2500 mV :

```C/C++
init{
    confAnalog(enable, 0b1110); // config des entrées analogiques
    confAnalogTomV(0, 5000);    // plage de 0V à 5V (0 à 5000 mV)
}
boucle{
    int valBinaire = analogRead(AN0);
    int tensionmV = analogTomV(valBinaire);
    if (tensionmV > 2500) {
        action();
    }
}
```

TMR0

Ce module traite du fonctionnement du timer 0.

void confTMR0(octet PS, unsigned short reset_delay, octet EN_int);
Configure le TMR0 et le lance.
Variables ->
PS alias Prescaler -> renseigne le binaire du prescaler
Voici un tableau des prescalers disponibles :

Nom de la macro	Valeur du prescaler
NPS_0	Pas de prescaler
PS2_0	1/2
PS4_0	1/4
PS8_0	1/8
PS16_0	1/16
PS32_0	1/32
PS64_0	1/64
PS128_0	1/128
PS256_0	1/256

Rappel des méthodes de choix :
Si on veut un timer de X ms :
PS = la valeur la plus proche au-dessus de 1000X / 65535

reset_delay = 65535 - 1000X * PS

void confTMR0ms(int ms, octet EN_int);
Configure le TMR0 en ms (limite : 16 000 ms).
void confTMR0us(long us, octet EN_int);
Configure le TMR0 en us.

Variables -> **ms / us** -> durée ; **EN_int** -> activation de l'interruption.

void resetTMR0(void);
Permet de réinitialiser le TMR0 et l'interruption si elle est utilisée.
startTMR0() : macro pour démarrer le TMR0 (T0CONbits.TMR0ON = 1).
stopTMR0() : macro pour arrêter le TMR0 (T0CONbits.TMR0ON = 0).

Exemple : on veut effectuer une tâche toutes les secondes :

```C/C++
init{
    confTMR0ms(1000, enable); // config toutes les secondes
    confINT(enable, disable, disable);
}
interrupt{
    if(TMR0IF){
        resetTMR0();
        tache();
    }
}
```

TMR1

Ce module traite du fonctionnement du timer 1.

void confTMR1(octet PS, unsigned short reset_delay, octet EN_int);
Configure le TMR1 et le lance.
Variables ->
PS alias Prescaler -> renseigne le binaire du prescaler
Voici un tableau des prescalers disponibles :

Nom de la macro	Valeur du prescaler
NPS_1	Pas de prescaler
PS2_1	1/2
PS4_1	1/4
PS8_1	1/8

Rappel des méthodes de choix :
Si on veut un timer de X µs (dans la limite de 524 000 µs) :
PS = la valeur la plus proche au-dessus de X / 65535

reset_delay = 65535 - X * PS

void confTMR1us(long us, octet EN_int);
Configure le TMR1 en us (limite : 524 000 µs).

Variables -> **us** -> durée en µs ; **EN_int** -> activation de l'interruption.

void resetTMR1(void);

Permet de réinitialiser le TMR1 et l'interruption si elle est utilisée.
startTMR1() : macro pour démarrer le TMR1 (T1CONbits.TMR1ON = 1).
stopTMR1() : macro pour arrêter le TMR1 (T1CONbits.T1RUN = 1).

Exemple : on veut effectuer une tâche toutes les 1000 µs :

```C/C++
init{
    confTMR1us(1000, enable); // config toutes les 1000 µs avec interruption
}
interrupt{
    if(TMR1IF){
        resetTMR1();
        tache();
    }
}
```

TMR2

Ce module traite du fonctionnement du timer 2.
Attention : *TMR2* est incompatible avec l'utilisation des modules **PWM** des **CCP**.

void confTMR2(octet PRS, octet POS, octet PR_Delay, octet reset_delay, octet EN_int);
Configure le TMR2 et le lance.
Variables ->
PRS alias Prescaler -> renseigne le binaire du prescaler
Voici un tableau des prescalers disponibles :

Nom de la macro	Valeur du prescaler
NPS_2	Pas de prescaler
PS4_2	1/4
PS16_2	1/16

POS alias Postscaler -> renseigne le binaire du postscaler (de POS1_2 à POS16_2 pour 1/1 à 1/16)
Voici un tableau des postscalers disponibles :

Nom de la macro	Valeur du postscaler
NPOS_2	1/1 (0b0000)
POS2_2	1/2 (0b0001)
POS3_2	1/3 (0b0010)
POS4_2	1/4 (0b0011)

Nom de la macro	Valeur du postscaler
POS5_2	1/5 (0b0100)
POS6_2	1/6 (0b0101)
POS7_2	1/7 (0b0110)
POS8_2	1/8 (0b0111)
POS9_2	1/9 (0b1000)
POS10_2	1/10 (0b1001)
POS11_2	1/11 (0b1010)
POS12_2	1/12 (0b1011)
POS13_2	1/13 (0b1100)
POS14_2	1/14 (0b1101)
POS15_2	1/15 (0b1110)
POS16_2	1/16 (0b1111)

PR_Delay -> limite maximale de TMR2
reset_delay ->
EN_int -> activation de l'interruption.

void confTMR2us(long us, octet EN_int);
Configure le TMR2 en us (limite : 65 500 µs).
Variables ->
us -> durée en µs ;
EN_int -> activation de l'interruption.
void resetTMR2(void);
Permet de réinitialiser le TMR2 et l'interruption si elle est utilisée.

startTMR2() : macro pour démarrer le TMR2 (T2CONbits.TMR2ON = 1).

stopTMR2() : macro pour arrêter le TMR2 (T2CONbits.TMR2ON = 0).

TMR3

Ce module traite du fonctionnement du timer 3.

void confTMR3(octet PS, unsigned short reset_delay, octet EN_int);
Configure le TMR3 et le lance.
Variables ->
PS alias Prescaler -> renseigne le binaire du prescaler
Voici un tableau des prescalers disponibles :

Nom de la macro	Valeur du prescaler
NPS_3	Pas de prescaler (1/1)
PS2_3	1/2
PS4_3	1/4
PS8_3	1/8

void confTMR3us(long us, octet EN_int);
Configure le TMR3 en us (limite : 524 000 µs).
Variables -> **us** -> durée en µs ; **EN_int** -> activation de l'interruption.
void resetTMR3(void);
Permet de réinitialiser le TMR3 et l'interruption si elle est utilisée.
startTMR3() : macro pour démarrer le TMR3 (T3CONbits.TMR3ON = 1).
stopTMR3() : macro pour arrêter le TMR3 (T3CONbits.TMR3ON = 0).

CCPX

Exemple de code complet pour un générateur PWM utilisant TMR0 et confPWM1 :
Exemple d'utilisation :

```C/C++
init{
    TRISA = 0xFF;
    TRISC = 0x00;
    confAnalog(enable, PCFG_A0_1_3);
    confTMR0ms(200, enable);
}
interrupt{
    if(TMR0IF) {
        resetTMR0();
        confPWM1((analogRead(AN1) + 1) * 4, analogRead(AN0));
    }
}
```

Exemple d'utilisation en mode capture :

```C/C++
init{
    TRISA = 0xFF;
    TRISC = 0xFF;
    confAnalog(enable, PCFG_A0_1_3);
    confTMR1us(250000, enable);
    confCapt1(CCP_capt_falling, enable);
}
interrupt{
    if(TMR1IF) {
        resetTMR1();
```

```
    }
    if(CCP1IF) {
        resetCCP1IF();
    }
}
```

aff7seg

Ce module s'occupe de l'affichage.

Câblage :
PORT1 <-> Led
PORT2[N:N+3] <-> dis[0:3]

Tris:
TRIS1 = 0x00;
TRIS2 = 0b…0000…
| |
N+3 N

void conf7seg(t_port *port, t_port *dis, octet pos_dis0);
Configure les ports du module.

Variables ->
port -> &PORT1
dis -> &PORT2
pos_dis0 -> N

octet convert7seg(octet chiffre);
Convertit **chiffre** et renvoie le code LED d'un 7 segments.

void write7seg(int val, octet points);
Rafraîchit les LED du 7 segments.

Variables :
val -> valeur à afficher ;
points -> bitmap de 4 bits pour placer les points

(VP) => **void conf7segVP(t_VirtualPort *port,t_VirtualPort *dis);**
Le 7 segments peut aussi être configuré via des VP, il faut conserver les VP car ils sont sauvegardés sous forme de pointeurs ;

Variables :
port -> VP avec lequel vous avez branché les LED

dis -> VP sur lequel vous avez mis les dis de 0 à 3 dans l'ordre

Exemple : afficher 10.24 sur un 7 segments :
Câblage :
PORTD <-> Led
PORTC[0:3] <-> dis[0:3]

```C/C++
init{
    TRISD = 0x00;
    TRISC = 0b0000;
    conf7seg(&PORTD, &PORTC, 0);
}
boucle{
    write7seg(1024, 0b0100);
}
```

KeyPad

Module permettant de lire un KeyPad.

Câblage :
PORT1 <-> KeyPad

Tris:
TRIS1 = 0x0f;

type **t_key_bitmap** est un bitmap des touches appuyées.

Touche	Macro	Binaire	Valeur Int
0	*KEY_0*	0b0000000000000010	0
1	*KEY_1*	0b0001000000000000	1
2	*KEY_2*	0b0010000000000000	2
3	*KEY_3*	0b0100000000000000	3

Touche	Macro	Binaire	Valeur Int
4	*KEY_4*	0b0000000100000000	4
5	*KEY_5*	0b0000001000000000	5
6	*KEY_6*	0b0000010000000000	6
7	*KEY_7*	0b0000000000010000	7
8	*KEY_8*	0b0000000000100000	8
9	*KEY_9*	0b0000000001000000	9
A	*KEY_A*	0b1000000000000000	10
B	*KEY_B*	0b0000100000000000	11
C	*KEY_C*	0b0000000010000000	12
D	*KEY_D*	0b0000000000001000	13
*	*KEY_Star*	0b0000000000000001	14
#	*KEY_Hashtag*	0b0000000000000100	15
Toutes	*all_KEY*	0b1111111111111111	*Default_KBTI*, cas null/plusieurs touches

Avec l'opérateur ~, on peut obtenir la négation. Ex. : *~KEY_Star* => toute touche sauf Star.
Default_KBTI est définie par défaut à 16.
(VP) => **void confKeyPadVP(t_VirtualPort *port);**
void confKeyPad(t_port *port);
Configure le port utilisé par le keypad pour la lecture.

t_key_bitmap keyPadRead();
Effectue la lecture d'un keypad et l'écrit au format bitmap.

return : le bitmap de l'ensemble des touches appuyées.

octet isKeySet(t_key_bitmap scan, t_key_bitmap key);
Renvoie si la touche **key** est active dans le **scan**.

Variables :
scan -> scan du read (le bouton appuyé) ou du change (le bouton a changé d'état)
key -> bitmap des touches à vérifier. Pour tester keyX ou keyY, vous pouvez mettre :
KEY_X | *KEY_Y* et il retournera si keyX ou keyY est actif.
Return :
un booléen indiquant si l'une des touches est appuyée ou non.

t_key_bitmap keyBitmapChange(t_key_bitmap scan);
Génère un *t_key_bitmap* pour les changements de touches depuis le dernier appel de la fonction.
Variables :
scan -> le scan le plus récent du keypad.

octet keyBitmapToInt(t_key_bitmap val);
Analyse val et renvoie un octet en fonction de la valeur de la touche. Si plusieurs touches sont pressées, il reste sur le cas par défaut défini par Default_KBTI dans PicP.h dans le module.
octet keyBitmapToIntPL(t_key_bitmap val);
Analyse val et renvoie la plus petite touche pressée.
octet keyBitmapToIntPG(t_key_bitmap val);
Analyse val et renvoie la plus grande touche pressée.

Sequential

Module permettant de récupérer une série de **t_Step** dans l'ordre et donc de réaliser des systèmes séquentiels.

Type **t_Step** : type modifiable pour les types de données récupérées et rendues par Sequential.
Par défaut, c'est **octet**.

Le #define *maxSeqStep* est le nombre maximum d'instructions dans une séquence.
Par défaut, c'est 16 instructions.

Type **t_sequence** : type de données qui permet de sauvegarder et de traiter une séquence.

void initSequence(t_sequence *seq);
Initialise le **t_sequence** de la séquence que l'on veut créer.
Variables :
seq -> le pointeur vers notre séquence

void addSequenceStep(t_sequence *seq, t_Step Step);
Ajoute l'étape **Step** à **seq** dans la limite de *maxSeqStep*.

Variables :
seq -> pointeur vers la séquence à modifier
Step -> t_Step lié à l'étape

t_Step SequenceStep(t_sequence *seq);
Lit l'instruction actuelle et passe la séquence **seq** à l'instruction suivante.

void SequenceSetPos(t_sequence *seq, octet pos);
Permet de définir la position **pos** dans la séquence **seq**.

LCD

Module permettant de piloter un afficheur LCD (nécessite le module Virtual_Port).
Macros de configuration LCD :
• **LCD_Command** (0b0) et **LCD_DATA** (0b1)
• **LCD_First_Command** (2)
• **LCDMoveCurseur** (0) et **LCDMoveAffichage** (1)
• **busy()** : attend que le LCD ne soit plus occupé
(while((LireTrameLCD(LCD_Command)&0b10000000)>0){})

octet LireTrameLCD(octet type);
Lit une trame depuis le LCD selon le type (commande ou donnée).

void EcrireTrameLCD(octet type, char trame);
Écrit une trame sur le LCD.

void confLCD(t_VirtualPort *DataPort, t_VirtualPort *ControlePort);
Configure et initialise le LCD avec les ports virtuels de données et de contrôle.

void printIntLCD(int val);
Affiche une valeur entière sur le LCD.

void clearLCD();
Efface l'affichage du LCD.

void gotoLCD(octet ligne, octet pos);
Déplace le curseur à la ligne et position spécifiées.

void printLCD(char* text);
Affiche une chaîne de caractères sur le LCD.

void confCurseurLCD(octet DisplayAll, octet CuseurDisplay, octet Blinking);
Configure les options d'affichage du curseur et du clignotement.

void moveLCD(octet typeMove, octet versLaDroite);
Déplace le curseur ou l'affichage vers la gauche ou la droite.